



**ITM SKILLS
UNIVERSITY**

School of Future Tech

Case Study Number 186 - Report

on

Ocean Pollution Tracking and Cleanup System

by

Abdeali Makda

150096725096

Cohort: Larry Page

Academic Year 2025–2029

Index

No.	Section	Page
1	Problem Statement.	3
2	Objectives.	4
3	Design/Architecture.	5
4	Algorithm Description.	7
5	Complexity Analysis.	8
6	Screenshots of Execution.	9
7	Results and Findings.	12
8	Conclusion.	12

1. Problem Statement

Marine pollution from oil spills, plastic waste, industrial discharge, and chemical contamination poses a serious threat to ocean ecosystems, biodiversity, and human health. Because ocean regions are interconnected by currents and shipping lanes, contamination originating in one region can spread rapidly into neighbouring waters before authorities are able to respond. Environmental agencies already deploy thousands of sensors across coastal and open-water regions that continuously measure pollution indicators such as chemical concentration, oil presence, and water quality, but raw sensor readings on their own do not tell responders which regions need attention first, how contamination is likely to spread, or how to get a cleanup vessel to the affected area as quickly as possible.

This project addresses that gap by building a console-based Ocean Pollution Tracking and Cleanup System that ingests live sensor telemetry in real time, ranks ocean regions by cumulative contamination severity, forecasts how contamination will propagate across connected regions, and computes the fastest route for a cleanup vessel to reach an affected region. The system is built entirely on four classical data structures and algorithms — a Queue, Merge Sort, Breadth-First Search, and Dijkstra's shortest-path algorithm — each chosen because its behavioural guarantees map directly onto a specific operational requirement of marine pollution response.

2. Objectives

The system was designed to meet the following objectives:

- **Reliable ingestion.** Track incoming marine pollution sensor readings in the exact order they are received, so that no telemetry packet is lost or processed out of sequence.
- **Severity-based prioritisation.** Rank ocean regions by cumulative contamination level so that cleanup effort can be directed at the most severely affected areas first.
- **Spread forecasting.** Predict how contamination will spread outward from an affected region into directly and indirectly connected regions.
- **Efficient routing.** Compute the fastest available route, in terms of travel time rather than hop count, for a cleanup vessel dispatched from its base to an affected region.
- **Auditable reporting.** Generate a persistent, file-based environmental report that can be archived or shared after each monitoring session.

3. Design / Architecture

System Overview

The system is implemented as a single C++ class, `OceanSystem`, that owns three core data members: a vector of `Region` records indexed directly by region ID for $O(1)$ lookups, a weighted adjacency list representing maritime travel lanes between regions, and a FIFO queue that buffers incoming sensor readings before they are committed to a region's cumulative pollution total. A menu-driven controller in `main()` lets the operator trigger ingestion, processing, ranking, spread forecasting, vessel routing, and report generation, redrawing a live status dashboard after every action so the current state of the ocean graph is always visible.

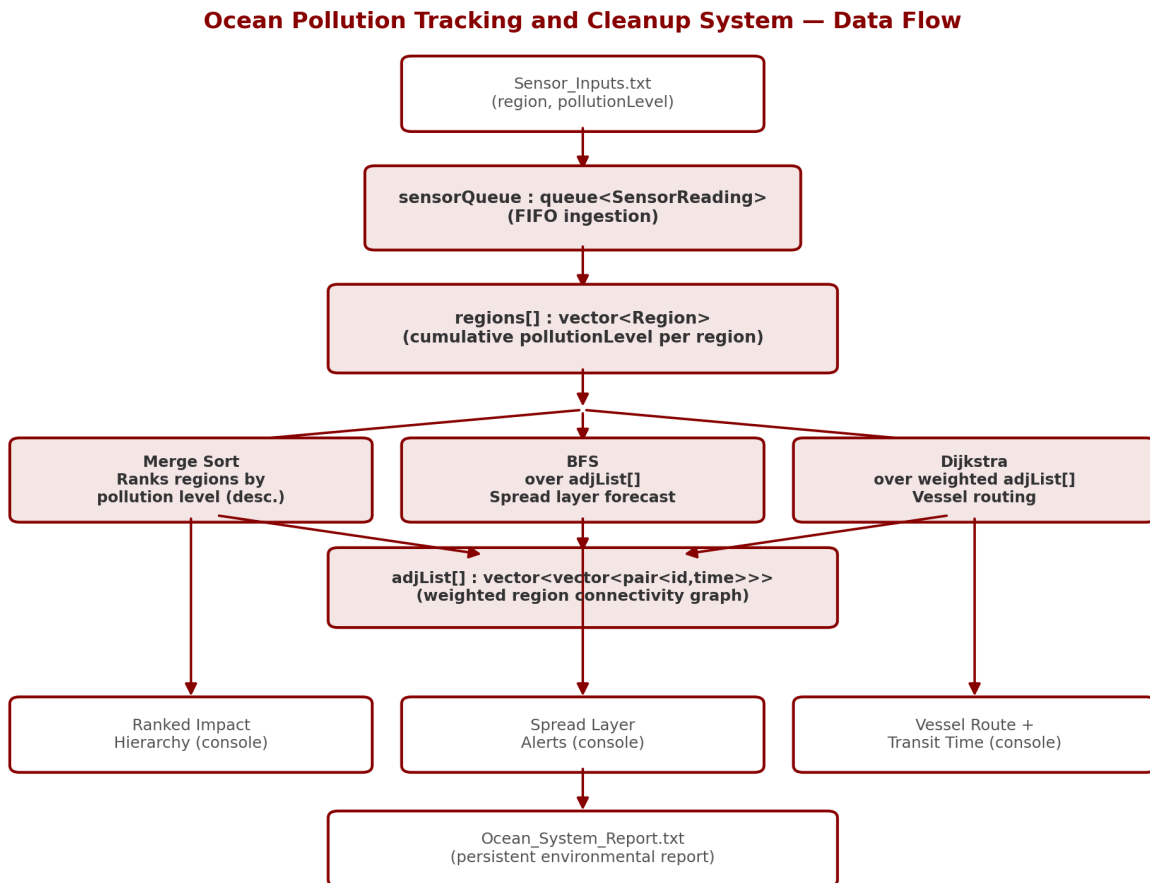


Figure 1. End-to-end data flow from sensor file ingestion through to ranking, spread forecasting, routing, and reporting.

Region Connectivity Model

Five ocean regions are modelled as nodes in a weighted, bidirectional graph: North Port (the cleanup vessel base), Coral Reef Matrix, Deep Oceanic Trench, Eastern Trade Gulf, and Industrial Delta. Edges represent maritime travel lanes between regions, weighted by travel time in hours. This same graph is reused for two different purposes: an unweighted traversal for contamination spread forecasting, and a weighted traversal for vessel route optimisation.

Maritime Region Connectivity Graph (edge labels = travel time, hours)

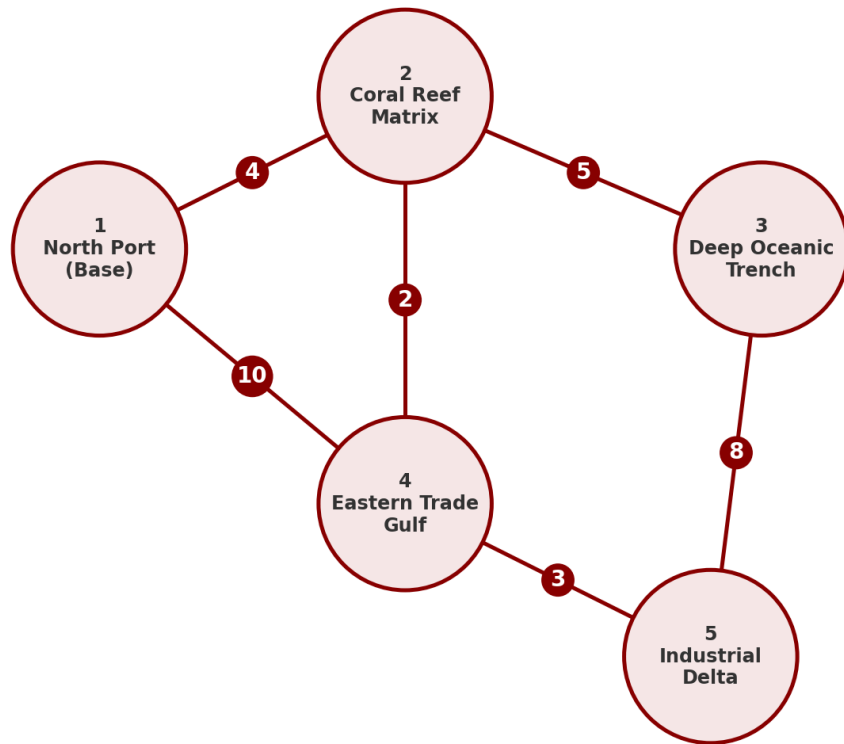


Figure 2. The five-region maritime connectivity graph used for both BFS spread detection and Dijkstra routing.

Core Data Structures

Member	Type	Responsibility
regions	vector<Region>	Stores each region's ID, name, and cumulative pollution level; indexed directly by region ID.
adjList	vector<vector<pair<int,int>>>	Weighted adjacency list of maritime connections; pair holds (neighbour ID, travel time).
sensorQueue	queue<SensorReading>	FIFO buffer holding telemetry packets awaiting processing.

4. Algorithm Description

4.1 Sensor Data Ingestion — Queue

Sensor readings loaded from `Sensor_Inputs.txt` are pushed onto a `std::queue<SensorReading>` in the order they appear in the file. Because a queue enforces strict First-In-First-Out ordering, `processSensorQueue()` always dispatches the oldest unprocessed reading first, which mirrors how real sensor telemetry must be handled: a reading transmitted earlier should never be superseded by a later one arriving out of order. Each dequeued reading's pollution level is added to its target region's cumulative total.

4.2 Pollution Ranking — Merge Sort

Once accumulated, regions are ranked by total contamination using a hand-written Merge Sort that recursively splits the region list in half, sorts each half, and merges them back together in descending order of pollution level. Merge Sort was chosen over an in-place comparison sort because it offers a guaranteed $O(n \log n)$ running time regardless of the input's initial order, and its stability means regions with equal pollution levels retain a predictable relative order — both useful properties when the same dataset is re-ranked repeatedly as new sensor data arrives.

4.3 Pollution Spread Detection — Breadth-First Search

`predictPollutionSpread()` runs a standard Breadth-First Search from a chosen source region over the unweighted connectivity graph. A queue of (regionId, layer) pairs ensures that all regions exactly one hop from the source are visited — and reported as Layer 1 — before any region two hops away is visited as Layer 2, and so on. This layer-by-layer expansion is exactly the behaviour required to forecast contamination spread: directly connected regions are at immediate risk, while regions reachable only through an intermediate region face a delayed, secondary risk.

4.4 Cleanup Vessel Routing — Dijkstra's Algorithm

`dispatchCleanupVessel()` computes the minimum-travel-time route from the vessel's base region to a target region using Dijkstra's algorithm with a binary-heap priority queue. Unlike the BFS used for spread detection, this traversal must account for edge weights (travel time in hours) rather than hop count, since the fastest route is not necessarily the route with the fewest intermediate stops. A parent array is maintained alongside the distance array so that, once the shortest distance to the target is found, the full navigation path can be reconstructed by walking backwards from the target to the source.

5. Complexity Analysis

Operation	Time Complexity	Space Complexity
Enqueue / dequeue a sensor reading	$O(1)$ per reading	$O(k)$ for k pending readings
Process entire queue	$O(k)$	$O(1)$ auxiliary
Merge Sort ranking	$O(n \log n)$	$O(n)$ auxiliary
BFS spread detection	$O(V + E)$	$O(V)$
Dijkstra shortest path	$O((V + E) \log V)$	$O(V)$
Report generation	$O(V)$	$O(1)$ auxiliary

Here V is the number of regions and E is the number of maritime connections between them; k is the number of sensor readings ingested in a given batch. With only five regions and six connections, every operation in this system completes well within real-time constraints even on a modest console application, but the chosen algorithms were selected specifically because their complexity bounds continue to hold as the dataset scales to the thousands of sensors and dozens of regions described in the original case study.

6. Screenshots of Execution

```

claude@ocean-sim: Options 1 & 2 - Queue Ingestion and Processing

$ ./ocean_pollution_system

=====
                LIVE OCEAN GRAPH MONITORING DASHBOARD
=====
REGIONAL CONTAMINATION LEVELS:
ID [1] North Port (Base)      : [-----] (0 Units)
ID [2] Coral Reef Matrix     : [-----] (0 Units)
ID [3] Deep Oceanic Trench   : [-----] (0 Units)
ID [4] Eastern Trade Gulf    : [-----] (0 Units)
ID [5] Industrial Delta      : [-----] (0 Units)
-----
LIVE INGESTION QUEUE STREAM:
[Empty] - No pending telemetry packets.
=====

Execute Selection Index: 1
[Success] Ingested 7 telemetry packets into the FIFO Queue.

Execute Selection Index: 2
--- Emptying Queue Pipeline (First-In, First-Out) ---
-> Dispatched Packet #7001 to North Port (Base)
-> Dispatched Packet #7002 to Coral Reef Matrix
-> Dispatched Packet #7003 to Deep Oceanic Trench
-> Dispatched Packet #7004 to Eastern Trade Gulf
-> Dispatched Packet #7005 to Industrial Delta
-> Dispatched Packet #7006 to North Port (Base)
-> Dispatched Packet #7007 to Eastern Trade Gulf

```

Figure 3. Menu options 1 and 2 — loading telemetry into the FIFO queue, then dispatching it into cumulative region totals.

```

=====
                        LIVE OCEAN GRAPH MONITORING DASHBOARD
=====
REGIONAL CONTAMINATION LEVELS:
ID [1] North Port (Base)      : [###-----] (35 Units)
ID [2] Coral Reef Matrix      : [###-----] (40 Units)
ID [3] Deep Oceanic Trench    : [#-----] (15 Units)
ID [4] Eastern Trade Gulf     : [#####-----] (80 Units)
ID [5] Industrial Delta       : [###-----] (30 Units)
-----
LIVE INGESTION QUEUE STREAM:
[Empty] - No pending telemetry packets.
=====

Execute Selection Index: 3
--- Priority Sorting Assessment (Merge Sort) ---
Ranked Impact Hierarchy:
1. Eastern Trade Gulf (Level: 80)
2. Coral Reef Matrix (Level: 40)
3. North Port (Base) (Level: 35)
4. Industrial Delta (Level: 30)
5. Deep Oceanic Trench (Level: 15)

```

Figure 4. Menu option 3 — regions ranked by descending contamination level using Merge Sort.

```

claude@ocean-sim: Options 4 & 5 – Spread Forecast (BFS) and Vessel Routing (Dijkstra)

Execute Selection Index: 4
Identify source spill incident point node (1-5): 1
--- Environmental Vector Expansion Forecasting (BFS) ---
[!] Spread Layer 1 Alert: Contamination expanding to -> Coral Reef Matrix
[!] Spread Layer 1 Alert: Contamination expanding to -> Eastern Trade Gulf
[!] Spread Layer 2 Alert: Contamination expanding to -> Deep Oceanic Trench
[!] Spread Layer 2 Alert: Contamination expanding to -> Industrial Delta

Press Enter to return to main dashboard menu...

Execute Selection Index: 5
Identify critical intervention site target (1-5): 5
--- Calculating Interconnected Travel Vectors (Dijkstra) ---
Vessel Vector Confirmed. Transit Estimate: 9 hours.
Navigation Leg: North Port (Base) -> Coral Reef Matrix -> Eastern Trade Gulf -> Industrial Delta

```

Figure 5. Menu options 4 and 5 — BFS spread-layer forecast from North Port, followed by the Dijkstra-routed vessel path to Industrial Delta.

```

claude@ocean-sim: Option 6 – Persistent Environmental Report

Execute Selection Index: 6
[Report] File compiled. Written safely to 'Ocean_System_Report.txt'

$ cat Ocean_System_Report.txt
=====
ECO-LOGISTICS MARITIME INTELLIGENCE REPORT
=====

Active Incidents Awaiting Processing Queue: 0

CURRENT POLLUTION METRICS BY REGION:
-----
Region ID [1] -> Name: North Port (Base) | Total Contamination Load: 35 units
Region ID [2] -> Name: Coral Reef Matrix | Total Contamination Load: 40 units
Region ID [3] -> Name: Deep Oceanic Trench | Total Contamination Load: 15 units
Region ID [4] -> Name: Eastern Trade Gulf | Total Contamination Load: 80 units
Region ID [5] -> Name: Industrial Delta | Total Contamination Load: 30 units

===== END OF REPORT =====

```

Figure 6. Menu option 6 — the persistent Ocean_System_Report.txt file written to disk.

7. Results and Findings

- **Ordering is preserved.** All seven packets in the sample run were dispatched in the exact order they were ingested (#7001 through #7007), confirming the queue preserves arrival order under processing.
- **Ranking reflects cumulative severity.** Eastern Trade Gulf was correctly ranked as the most contaminated region (80 units) after receiving two separate sensor readings, showing that ranking reflects cumulative load rather than a single reading.
- **Spread layering matches topology.** BFS correctly distinguished directly connected regions (Layer 1: Coral Reef Matrix, Eastern Trade Gulf) from regions reachable only through an intermediate hop (Layer 2: Deep Oceanic Trench, Industrial Delta), matching the graph's actual connectivity.
- **Weighted routing outperforms hop-count routing.** Dijkstra routed the vessel via North Port → Coral Reef Matrix → Eastern Trade Gulf → Industrial Delta (9 hours) rather than the direct-looking North Port → Eastern Trade Gulf → Industrial Delta path, because it correctly accounts for edge weights rather than hop count — the shorter-looking route is actually slower once travel times are summed.
- **Edge cases are handled gracefully.** The system handles an empty queue, an unreachable target region, and invalid menu input without crashing, after bounds-checking was added to every method that accepts a user-supplied region ID.

8. Conclusion

This project demonstrates how four classical data structures and algorithms map directly onto a real-world environmental monitoring problem. A Queue enforces correct temporal ordering of incoming sensor data; Merge Sort reliably prioritises cleanup effort by contamination severity; Breadth-First Search models how pollution propagates across connected ocean regions; and Dijkstra's algorithm finds the most time-efficient cleanup vessel routes across a weighted maritime network. Together, wrapped in a live monitoring dashboard and persistent file-based reporting, these four components fulfil the case study's stated goal of continuous monitoring and coordinated, data-driven cleanup response, while remaining simple enough to extend to a larger number of regions and sensors without any change to the underlying algorithmic approach.

Project Repository:

[Github](#)

Declaration

I hereby declare that this case study report on Ocean Pollution Tracking and Cleanup System is an original work completed by me. All external references and resources have been duly cited. The application, its code, and the analysis presented in this report are my own work unless otherwise stated.

Abdeali Makda

Roll No: 150096725096

Cohort: Larry Page

School of Future Tech, ITM Skills University

End of Report — Ocean Pollution Tracking and Cleanup System | Abdeali Makda